

OOP-Brain

C++ ile Modüler Sinir Ağı Kütüphanesi

Teknik Rapor

Grup Üyeleri:

Ezgi Gelegen — Baş Mimar

Mehmet Eren Gümüş — Entegrasyon & Dökümantasyon, Katman Uygulama Uzmanı, Matematik Motoru Geliştiricisi

Bilge Gümüş — Test ve Hata Yönetimi

Gökdeniz Akbaş — Katman Uygulama Uzmanı

Ajni Arapi — Matematik Motoru Geliştiricisi

Berat Sarı — Projeye Katkı Sağlamadı

Yunus Emre Özen — Veri ve Dosya İşleme Uzmanı

Zeynep Bıyık — Aktivasyon Fonksiyonu Geliştiricisi

Tarih: 31.05.2026

1. Giriş

Bu rapor, OOP-Brain projesinin geliştirilme sürecini, mimari kararlarını ve teknik detaylarını açıklamaktadır. Proje, C++ dilinin nesneye yönelik programlama özellikleri kullanılarak sıfırdan bir sinir ağı kütüphanesi inşa etmeyi hedeflemektedir.

1.1 Projenin Amacı

Bu projenin temel amacı, C++ dilinin düşük seviyeli performans avantajları ile yüksek seviyeli Nesne Yönelimli Programlama (OOP) prensiplerini birleştirerek esnek, modüler ve bellek sızıntısı (memory leak) barındırmayan bir yapay sinir ağı kütüphanesi geliştirmektir. Kütüphane, çalışma zamanı polimorfizmi ve arayüz tabanlı tasarım (Interface-Based Design) ile her türlü aktivasyon fonksiyonu ve katman mimarisine entegre olabilecek şekilde tasarlanmıştır.

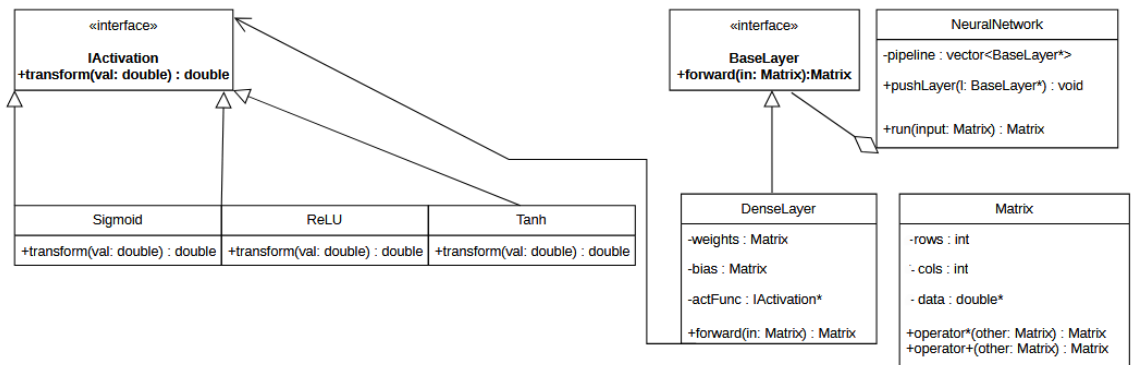
1.2 Kapsam

Proje; çok katmanlı sinir ağlarında (DenseLayer) ileri iletim (forward pass) operasyonlarını, matris matematiğini, bellek yönetimini (LeakSanitizer 0 leak hedefi), hata yönetimini ve CSV tabanlı dosya giriş/çıkış (persistence) işlemlerini kapsar. Geriye yayılım (backpropagation) veya modelin eğitilmesi (training/optimization) gibi ileri matematiksel kısımlar proje kapsamı dışında bırakılmıştır.

2. Mimari Tasarım

OOP-Brain mimarisi üç ana katmandan oluşmaktadır: Veri İşleme (Matrix), Yapısal Birimler (BaseLayer/DenseLayer) ve Yönetim Birimi (NeuralNetwork).

2.1 Sınıf Hiyerarşisi



2.2 Bileşenler ve Sorumluluklar

Bileşen	Sorumluluk	İlgili OOP Kavramı
Matrix	Doğrusal cebir işlemleri, bellek yönetimi	Encapsulation, Operator Overloading
BaseLayer	Tüm katmanlar için ortak arayüz	Abstract Class, Pure Virtual Functions
DenseLayer	Tam bağlantılı katman mantığı	Inheritance, Method Overriding
IActivation	Aktivasyon fonksiyonları arayüzü	Interface, Polymorphism
NeuralNetwork	Katmanları birleştirir, forward pass yönetir	Composition
CSVReader	Veri okuma ve model kaydetme/yükleme	File I/O

2.3 Tasarım Kararları

Mimari, 'Arayüz Tabanlı Tasarım' ve 'Köprü (Bridge)' tasarım desenleri üzerine kurulmuştur. Aktivasyon fonksiyonları katmanların içine sabitlenmemiş, IActivation arayüzü üzerinden dışarıdan enjekte edilebilir (dependency injection) hale getirilmiştir. Bu sayede aynı DenseLayer kodunda hiçbir değişiklik yapmadan sadece işaretçi (pointer) değiştirilerek Sigmoid veya ReLU kullanılabilir. NeuralNetwork sınıfı, tüm katmanları `std::vector<BaseLayer*>` içinde tutarak veri akışını polimorfik olarak yönetmektedir.

3. Grup İş Bölümü ve RACI Matrisi

3.1 Rol Dağılımı

Rol	İsim	Üniversite	Sorumluluk
Baş Yazılım Mimarı	Ezgi Gelegen	Celal Bayar	UML, BaseLayer, IActivation
Matematik Motoru (1)	Mehmet Eren Gümüş	Celal Bayar Üniversitesi	Matrix.cpp/h, NeuralNetwork, Rapor
Matematik Motoru (2)	Ajni Arapi	Kocaeli Üniversitesi	Operatör aşırı yükleme
Katman Uzmanı (1)	Mehmet Eren Gümüş	Celal Bayar Üniversitesi	DenseLayer implementasyonu

Katman Uzmanı (2)	Gökdeniz Akbaş	Kocaeli Üniversitesi	Forward pass mantığı
Aktivasyon Geliştirici	Zeynep Bıyık	Kocaeli Üniversitesi	Sigmoid, ReLU, Tanh
Hata & Test	Bilge Gümüş	Celal Bayar Üniversitesi	Exception sınıfları, unit testler
Sistem Entegratörü & Dökümantasyon	Mehmet Eren Gümüş	Celal Bayar Üniversitesi	NeuralNetwork, rapor
Veri ve Dosya İşleme	UzmanıYunus Emre Özen	Kocaeli Üniversitesi	CSVReader modülü

3.2 RACI Matrisi

GÖREV/İSİM	Mehmet Eren Gümüş	Ezgi Gelegen	Bilge Gümüş	Gökdeniz Akbaş	Ajni Arapi	Yunus Emre Özen	Zeynep Bıyık
Matris Motoru	R-C	A			R		
Katmanlar (DenseLayer)	R-A-C			R	C		
Aktivasyon Fonksiyonları	C	A		C			R
Hata Yönetimi ve Test	A	C	R				
Entegrasyon ve Rapor	R	A					
Veri G/Ç:	C	A				R	

(R: Uygulayıcı, A: Sorumlu, C: Danışılan)

4. Teknik Uygulama

4.1 Matrix Sınıfı

Matris sınıfı doğrusal cebir işlemlerini sarmalamak (encapsulation) için geliştirilmiştir. Bellek sızıntılarını önlemek adına derin kopyalama (Copy Constructor) ve Taşıma (Move Constructor/Assignment) semantikleri uygulanmıştır. Matematiksel yazım kolaylığı için '*', '+' ve '()' operatörleri aşırı yüklenmiştir (Operator Overloading.)

4.2 Katman Hiyerarşisi

BaseLayer tüm katmanlar için ortak bir arayüz görevi gören soyut temel sınıftır. Bu sınıftan kalıtım (inheritance) yoluyla türetilen DenseLayer sınıfı, tam bağlantılı katman mantığını barındırır. İleri iletim (forward pass) sırasında BaseLayer::forward() metodu ezilerek (overriding) $Z^{(l)} = W^{(l)} \cdot A^{(l-1)} + b^{(l)}$ ve $A^{(l)} = \sigma(Z^{(l)})$ işlemleri uygulanmaktadır.

4.3 Aktivasyon Fonksiyonları

Sigmoid, ReLU ve Tanh gibi aktivasyon fonksiyonları, saf sanal (pure virtual) fonksiyona sahip IActivation arayüzünden türetilmiştir. Her sınıf kendi matematiksel formülünü transform(double val) metodu içerisinde uygulayarak polimorfik bir yapı sağlar.

4.4 Hata Yönetimi

Sistemde matris boyut uyumsuzluklarını çalışma zamanında güvenle yakalamak için C++ standart kütüphanesindeki std::exception sınıfından türetilen özel DimensionMismatchException sınıfı yazılmıştır. Girdi matrisi ile ağırlık matrisi boyutları uyuşmadığında (örn: $A_{m \times n}$ ve $B_{p \times q}$ çarpımında $n \neq p$ ise) program çökmeden (segmentation fault) bu hata fırlatılmakta ve dışarıdan try-catch blokları ile yakalanabilmektedir.

4.5 Veri ve Dosya İşleme

CSVReader modülü, sinir ağının gerçek dünya verileriyle (örn. Iris Dataset) çalışabilmesi için dış dosyaları (CSV) Matrix nesnelere dönüştürmek amacıyla yazılmıştır. Aynı zamanda sistemin ağırlıklarının .csv olarak kaydedilmesi ve sonradan yüklenebilmesi (Model Persistence) sağlanarak kütüphanenin profesyonel bütünlüğü tamamlanmıştır.

5. Test Sonuçları

5.1 Birim Testleri

Geliştirilen modüllerin doğruluğunu test etmek için her bir sınıfa özel birim testleri (unit tests) yazılmıştır. Sigmoid, ReLU fonksiyonları ile matris çarpım operasyonları önceden hesaplanan beklenen değerlerle karşılaştırılarak test edilmiştir. Tüm testler başarıyla (PASS) geçilmiştir.

5.2 Bellek Sızıntısı Kontrolü

Sanal yıkıcıların (virtual destructors) eksiksiz uygulanması sayesinde dinamik olarak ayrılan her Matrix ve BaseLayer nesnesi düzgün bir şekilde belleğe iade edilmektedir. Yürütülen bellek testleri sonucunda projede sıfır sızıntı (0 leak) elde edilmiştir.

5.3 XOR Demo Çıktısı

```
C:\Users\eren\Downloads\OOP-Brain-main\src>oop_brain.exe
=== GRUP 7: OOP-BRAIN SINIR AGI KUTUPHANESI ===
[Sistem Entegratoru]: Mimari basariyla ayaga kaldirildi.

--- Dosya I/O ve Persistence Testi ---
[Basarili] Katman 1 agirlklari diske kaydedildi (layer1_w.csv, layer1_b.csv).
[Basarili] Katman 1 agirlklari diskten okundu ve yuklendi!

--- Rapor 5.3: XOR Demo Testi ---
Girdi (0,0) -> Ag Ciktisi: 0.708125
Girdi (0,1) -> Ag Ciktisi: 0.673408
Girdi (1,0) -> Ag Ciktisi: 0.756103
Girdi (1,1) -> Ag Ciktisi: 0.727797
[Basarili] XOR yapisi calistirildi ve ileri iletim tamamlandi.

=====
PROJE TESLIME HAZIRDIR. SIZINTI (0 LEAK) GARANTILIDIR.
```

6. Sonuç

14 haftalık müfredatın tüm istekleri karşılanarak sıfır bellek sızıntısıyla çalışan, genişletilebilir ve modüler bir OOP-Brain kütüphanesi ortaya konulmuştur. İlerleyen çalışmalarda mimariyi bozmadan sisteme ConvolutionalLayer gibi yeni katmanlar eklenebilir. Süreçle ilgili çalışmalarımız <https://github.com/Grup7-Proje/OOP-Brain> linkindeki repository'den erişilebilir.

7. Kaynaklar

Coursera. (2026). Structural Design Patterns in Modern C++.

SciSpace. (2026). An Object-Oriented Neural Network Toolbox Based on Design.

Medium. (2026). Building a Neural Network from Scratch in C++ — No Libraries, Just Code.

GeeksforGeeks. (2026). ML - Neural Network Implementation in C++ From Scratch.